

小红书前端技术专家-社区工程

一面（技术基础） · 1-9

1. 请简述一下CSS 盒模型，以及 box-sizing 属性对它的影响。
2. 请解释一下 JavaScript 的事件循环（Event Loop）机制。
3. 谈谈你对闭包的理解，以及它在实际开发中的应用场景。
4. Promise 有哪几种状态？Promise.all 和 Promise.allSettled 有什么区别？
5. 请手写一个防抖（debounce）函数。
6. 什么是原型链？当我们访问一个对象的属性时，查找过程是怎样的？
7. 数组去重有哪些实现方式？请至少说出三种。
8. 简单介绍一下 HTTP 缓存策略（强缓存与协商缓存）。
9. 算法题：给定一个字符串，找出其中无重复字符的最长子串的长度。

二面（深入原理与项目） · 1-8

1. React 的 Fiber 架构解决了什么问题？它的工作原理是怎样的？
2. 谈谈你对 React Hooks 的理解，为什么不能在循环或条件语句中使用Hooks？
3. 你们项目里是如何进行前端性能优化的？有哪些量化指标？
4. Webpack 的构建流程是怎样的？你做过哪些构建速度的优化？
5. 什么是微前端？在什么场景下会使用，常见的实现方案有哪些？
6. 请实现一个深拷贝函数，需要考虑循环引用和特殊类型（如 Map、Set、RegExp）。
7. 谈谈你对跨域的理解，生产环境下通常怎么解决跨域问题？
8. 如果让你设计一个虚拟列表（Virtual List），你的核心思路是什么？

三面（架构与综合能力） · 1-6

1. 如果让你从零搭建一个前端监控系统，你会如何设计它的架构？
2. 在过去的项目中，你遇到的最具有挑战性的技术问题是什么？你是如何解决的？
3. 聊聊你对前端工程化的理解，你认为目前团队最需要提升的工程化环节是什么？
4. 面对一个大型遗留项目（Legacy Project），如果需要技术栈升级，你会制定怎样的策略？
5. 你是如何保持技术竞争力的？最近在关注哪些前端领域的新趋势？
6. 谈谈你对前端团队协作和 Code Review 的看法。

一面（技术基础）

1. 请简述一下 CSS 盒模型，以及 box-sizing 属性对它的影响。

难度：★

考点：CSS 布局基础

答案要点：

盒模型是浏览器对 HTML 元素进行布局的基础，它规定了元素占用的空间由内容、内边距、边框和外边距组成。

- 标准盒模型：width 仅包含 content，不包含 padding 和 border。
- 怪异盒模型：width 包含 content、padding 和 border。
- box-sizing: content-box 对应标准盒模型，是浏览器默认值。
- box-sizing: border-box 对应怪异盒模型，在响应式布局中更易于计算尺寸。

常见坑：

- 混淆外边距合并（Margin Collapsing）与盒模型尺寸计算的关系。
- 忽略内联元素（inline）在盒模型表现上的特殊性。

追问：

- 如何解决外边距重叠问题？
- 什么是 BFC，它如何影响盒模型？

2. 请解释一下 JavaScript 的事件循环（Event Loop）机制。

难度：★★

考点：JS 异步执行机制

答案要点：

事件循环是 JavaScript 处理异步操作的核心机制，通过任务队列的调度实现非阻塞 I/O。

- 执行栈：同步代码按顺序入栈执行，执行完毕后出栈。
- 微任务（Microtask）：包括 Promise.then、MutationObserver，在当前宏任务结束后立即执行。
- 宏任务（Macrotask）：包括 setTimeout、setInterval、I/O，在微任务队列清空后执行。
- 渲染时机：浏览器通常在微任务清空后、下一个宏任务开始前尝试进行页面渲染。

常见坑：

- 错误地认为 setTimeout(fn, 0) 会立即执行。
- 不清楚 async/await 在事件循环中的转换逻辑。

追问：

- Node.js 的事件循环与浏览器有什么区别？

- 如果微任务里不断产生新的微任务，会发生什么？

3. 谈谈你对闭包的理解，以及它在实际开发中的应用场景。

难度：★★

考点：作用域与内存管理

答案要点：

闭包是指有权访问另一个函数作用域中变量的函数，本质是函数与其词法环境的引用捆绑。

- 形成条件：函数嵌套且内部函数引用了外部函数的变量。
- 核心作用：实现数据的私有化，防止全局变量污染。
- 内存影响：由于闭包持有外部变量引用，可能导致相关内存无法被垃圾回收。
- 典型应用：防抖节流函数、模块化封装、柯里化（Currying）。

常见坑：

- 在循环中使用 var 定义变量导致的闭包陷阱。
- 滥用闭包导致内存泄漏而未手动解除引用。

追问：

- 现代浏览器是如何优化闭包内存占用的？
- 闭包和匿名函数是一个概念吗？

4. Promise 有哪几种状态？Promise.all 和 Promise.allSettled 有什么区别？

难度：★★

考点：异步编程 API

答案要点：

Promise 是异步编程的一种解决方案，具有三种状态：Pending（进行中）、Fulfilled（已成功）和 Rejected（已失败）。

- 状态特性：状态一旦改变就不可逆转。
- Promise.all：所有任务都成功才返回成功结果，只要有一个失败就立即返回失败。
- Promise.allSettled：等待所有任务结束（无论成功或失败），返回包含每个任务状态和结果的数组。
- 错误处理：Promise.all 适合有强依赖关系的请求，allSettled 适合互不影响的并发任务。

常见坑：

- 忘记在 Promise 中写 reject 导致状态一直处于 pending。
- 在 Promise.all 中没有对单个 promise 做 catch 处理导致整个链路中断。

追问：

- 如何手写实现一个 Promise.race？
- 如果 Promise.all 中某个请求特别慢，会影响其他请求的执行吗？

5. 请手写一个防抖（debounce）函数。

难度：★★

考点：高阶函数、定时器应用

答案要点：

防抖是指在事件被触发 n 秒后再执行回调，如果在这 n 秒内又被触发，则重新计时。

- 核心逻辑：利用闭包存储定时器变量 timer。
- 清除机制：每次触发时先 clearTimeout。
- 延迟执行：通过 setTimeout 在指定时间后执行目标函数。
- 参数处理：需考虑 this 指向及 arguments 参数传递。

常见坑：

- 忘记处理 this 指向问题。
- 频繁创建闭包导致性能开销。

代码示例：

代码块

```
1 function debounce(fn, delay) {  
2   let timer = null;  
3   return function(...args) {  
4     if (timer) clearTimeout(timer);  
5     timer = setTimeout(() => {  
6       fn.apply(this, args);  
7     }, delay);  
8   };  
9 }
```

追问：

- 如何给防抖函数增加一个“立即执行”的选项？

6. 什么是原型链？当我们访问一个对象的属性时，查找过程是怎样的？

难度：★★

考点：原型继承机制

答案要点：

原型链是 JavaScript 实现继承的机制，每个对象都有一个指向其原型对象的内部链接。

- 查找逻辑：先在对象自身属性查找，找不到则顺着 **proto** 向上级原型查找。
- 终点：原型链的顶端是 **Object.prototype.proto**，其值为 null。
- 构造函数：函数的 **prototype** 属性指向其实例的原型。

- 性能影响：过长的原型链会导致属性查找耗时增加。

常见坑：

- 混淆 prototype 和 **proto**。
- 修改原型对象属性时对所有实例产生的影响。

追问：

- 如何判断一个属性是对象自身的还是原型链上的？
- ES6 的 class 继承在底层是如何通过原型链实现的？

7. 数组去重有哪些实现方式？请至少说出三种。

难度：★

考点：数据结构操作

答案要点：

数组去重是基础的数据处理需求，可以通过 ES6 新特性或传统循环实现。

- Set 结构：Array.from(new Set(arr))，最简洁高效。
- Map/对象键值对：利用 key 的唯一性，适合处理包含复杂类型的数组。
- filter + indexOf：通过判断当前索引是否为第一次出现的索引。
- 性能对比：Set 和 Map 在大数据量下表现优于嵌套循环。

常见坑：

- Set 去重无法处理引用类型（如两个内容相同的对象）。
- 使用 indexOf 在 NaN 处理上的缺陷。

追问：

- 如果数组元素是嵌套对象，如何实现深度去重？

8. 简单介绍一下 HTTP 缓存策略（强缓存与协商缓存）。

难度：★★

考点：网络协议与性能

答案要点：

HTTP 缓存通过减少重复请求来提升页面加载速度，分为强缓存和协商缓存。

- 强缓存：浏览器直接从本地缓存读取，不发送请求。涉及字段 Cache-Control (max-age) 和 Expires。
- 协商缓存：浏览器发送请求到服务器，由服务器决定是否使用缓存。涉及字段 ETag/If-None-Match 和 Last-Modified/If-Modified-Since。
- 优先级：Cache-Control 优先级高于 Expires；ETag 优先级高于 Last-Modified。
- 状态码：强缓存命中返回 200 (from disk/memory cache)，协商缓存命中返回 304。

常见坑：

- 误以为 304 也会传输完整的响应体。
- 不清楚 Ctrl+F5 强制刷新对缓存字段的影响。

追问：

- 为什么 ETag 比 Last-Modified 更可靠？
- 什么是启发式缓存？

9. 算法题：给定一个字符串，找出其中无重复字符的最长子串的长度。

难度：★★

考点：滑动窗口、字符串处理

答案要点：

该问题通常使用滑动窗口（双指针）配合哈希表来解决。

- 窗口维护：左指针 start 和右指针 end 定义当前无重复子串范围。
- 哈希表记录：记录字符最后出现的位置，用于快速移动左指针。
- 动态更新：遍历字符串，遇到重复字符时将 start 移至重复位置的下一位。
- 结果记录：每次移动右指针后更新最大长度。

代码示例：

代码块

```
1  function lengthOfLongestSubstring(s) {
2    let map = new Map();
3    let maxLen = 0, start = 0;
4    for (let end = 0; end < s.length; end++) {
5      if (map.has(s[end])) {
6        start = Math.max(map.get(s[end]) + 1, start);
7      }
8      map.set(s[end], end);
9      maxLen = Math.max(maxLen, end - start + 1);
10   }
11   return maxLen;
12 }
```

追问：

- 空间复杂度是多少？能否优化？

二面（深入原理与项目）

1. React 的 Fiber 架构解决了什么问题？它的工作原理是怎样的？

难度：★★★

考点：React 核心原理

答案要点：

Fiber 是 React 16 引入的新的协调引擎，主要解决了长任务阻塞主线程导致的页面卡顿问题。

- 核心痛点：传统的 Stack Reconciler 是递归执行，无法中断，会导致大组件树更新时掉帧。
- 任务拆分：将更新过程拆分为多个小单元（Fiber），支持异步渲染。
- 优先级调度：通过 Scheduler 为不同任务分配优先级（如用户输入高于数据请求）。
- 双缓存技术：维护 current 树和 workInProgress 树，在内存中构建好后再统一提交到 DOM。

常见坑：

- 误认为 Fiber 提升了单次计算的速度（实际上增加了计算量，但提升了响应性）。
- 不清楚哪些生命周期在 Fiber 架构下会被多次触发。

追问：

- requestIdleCallback 在 Fiber 中起到了什么作用？
- 为什么 React 不直接使用 Generator 来实现中断？

2. 谈谈你对 React Hooks 的理解，为什么不能在循环或条件语句中使用 Hooks？

难度：★★

考点：Hooks 运行机制

答案要点：

Hooks 允许在函数组件中使用状态和其他 React 特性，解决了类组件逻辑复用难和生命周期分散的问题。

- 存储机制：React 在内部为每个组件维护一个 Fiber 节点，Hooks 以链表形式按顺序存储在 Fiber 的 memoizedState 属性中。
- 顺序依赖：React 依赖 Hooks 的调用顺序来关联状态，如果顺序改变，状态对应就会出错。
- 闭包陷阱：useEffect 或 useCallback 捕获的是定义时的变量快照，需通过依赖数组更新。
- 逻辑复用：通过自定义 Hooks 可以将 UI 逻辑与业务逻辑完美分离。

常见坑：

- 在 useEffect 中使用了外部变量却没将其放入依赖数组。
- 忽略了 Hooks 的闭包特性导致的“陈旧闭包”问题。

追问：

- useMemo 和 useCallback 的区别是什么？
- 如何在不重新渲染组件的情况下获取最新的 props 值？

3. 你们项目里是如何进行前端性能优化的？有哪些量化指标？

难度：★★★

考点：性能优化工程化

答案要点：

性能优化应从加载性能和运行时性能两个维度展开，并建立监控体系。

- 加载优化：代码分割（Code Splitting）、静态资源 CDN、Gzip/Brotli 压缩、图片懒加载及 WebP 转换。
- 运行时优化：减少重排重绘、虚拟列表（Virtual List）、防抖节流、React.memo/useMemo。
- 关键指标：LCP（最大内容绘制）、FID（首次输入延迟）、CLS（累积布局偏移）。
- 监控工具：使用 Lighthouse 进行离线分析，接入 Web Vitals 进行线上数据打点。

常见坑：

- 盲目进行优化而没有数据支撑。
- 优化了非关键路径的代码，导致投入产出比低。

追问：

- 什么是“时间分片”（Time Slicing）？
- 如何优化单页应用（SPA）的首屏加载速度？

4. Webpack 的构建流程是怎样的？你做过哪些构建速度的优化？

难度：★★

考点：构建工具原理

答案要点：

Webpack 是一个静态模块打包器，其过程可以概括为初始化、编译、输出三个阶段。

- 核心流程：读取配置 -> 实例化 Compiler -> 确定入口 -> AST 解析依赖 -> Loader 转换 -> Plugin 触发钩子 -> 生成 Chunk -> 输出文件。
- 速度优化：使用持久化缓存（cache: { type: 'filesystem' }）、多进程并行构建（thread-loader）。
- 体积优化：Tree Shaking 剔除死代码、DllPlugin（虽然 Webpack 5 不再推荐）、代码压缩。
- 调试工具：speed-measure-webpack-plugin 分析各阶段耗时。

常见坑：

- Loader 作用范围（include/exclude）配置不当导致全量扫描 node_modules。
- 滥用 Plugin 导致构建过程产生大量不必要的计算。

追问：

- Loader 和 Plugin 的区别是什么？
- Vite 为什么比 Webpack 快？

5. 什么是微前端？在什么场景下会使用，常见的实现方案有哪些？

难度：★★★

考点：架构设计

答案要点：

微前端是一种将巨型应用拆分为多个独立交付的小型前端应用的设计理念。

- 核心价值：技术栈无关、独立开发部署、增量升级旧系统。
- 隔离机制：样式隔离（Shadow DOM、Scoped CSS）、JS 沙箱（Proxy 代理全局对象）。
- 常见方案：iframe（最简单但体验差）、single-spa、qiankun（基于 single-spa 封装）、Module Federation（Webpack 5）。
- 通信方式：CustomEvent、Props 传递、全局状态管理。

常见坑：

- 忽略了微前端带来的维护成本和公共依赖提取的复杂度。
- 多个子应用同时运行时的内存占用问题。

追问：

- qiankun 是如何实现 JS 沙箱的？
- 微前端架构下如何处理全局路由跳转？

6. 请实现一个深拷贝函数，需要考虑循环引用和特殊类型（如 Map、Set、RegExp）。

难度：★★★★

考点：递归与复杂数据处理

答案要点：

深拷贝需要递归遍历对象，并处理各种边缘情况以保证数据的完整性。

- 循环引用：使用 WeakMap 存储已拷贝的对象，发现重复引用直接返回。
- 类型识别：利用 Object.prototype.toString.call() 精确判断类型。
- 特殊处理：针对 Map、Set 需手动遍历赋值；针对 RegExp、Date 需重新构造实例。
- 性能考虑：优先处理基本类型，减少递归深度。

代码示例：

代码块

```
1 function deepClone(target, map = new WeakMap()) {
2   if (target === null || typeof target !== 'object') return target;
3   if (target instanceof Date) return new Date(target);
4   if (target instanceof RegExp) return new RegExp(target);
5
6   if (map.has(target)) return map.get(target);
7
8   const cloneTarget = Array.isArray(target) ? [] : {};
9   map.set(target, cloneTarget);
```

```
10
11     if (target instanceof Map) {
12         target.forEach((v, k) => cloneTarget.set(k, deepClone(v, map)));
13     } else if (target instanceof Set) {
14         target.forEach(v => cloneTarget.add(deepClone(v, map)));
15     } else {
16         Reflect.ownKeys(target).forEach(key => {
17             cloneTarget[key] = deepClone(target[key], map);
18         });
19     }
20     return cloneTarget;
21 }
```

追问：

- 为什么使用 WeakMap 而不是 Map？

7. 谈谈你对跨域的理解，生产环境下通常怎么解决跨域问题？

难度：★★

考点：浏览器安全策略

答案要点：

跨域是由浏览器的同源策略引起的，限制了不同源（协议、域名、端口）之间的资源交互。

- CORS：最主流方案，服务器设置 Access-Control-Allow-Origin 等响应头。
- 反向代理：在生产环境常用 Nginx 转发请求，让前端和接口看起来同源。
- JSONP：利用 script 标签不受同源策略限制的特性，但仅支持 GET 请求。
- PostMessage：用于不同窗口或 iframe 之间的跨域通信。

常见坑：

- 简单请求与预检请求（Preflight）的区别，不清楚为什么会多发一个 OPTIONS 请求。
- 跨域时 Cookie 无法携带的问题（需设置 withCredentials）。

追问：

- Nginx 转发跨域的原理是什么？
- 什么是 CSRF 攻击，它和跨域有什么关系？

8. 如果让你设计一个虚拟列表（Virtual List），你的核心思路是什么？

难度：★★★

考点：长列表性能优化

答案要点：

虚拟列表通过只渲染可视区域内的 DOM 节点，来解决海量数据下的渲染性能问题。

- 容器结构：外层固定高度容器（overflow: auto），内层撑开总高度的占位区。

- 动态计算：根据滚动距离 scrollTop 计算当前起始索引 startIndex 和结束索引 endIndex。
- 偏移处理：计算当前渲染列表相对于顶部的偏移量 transform: translateY，保持在可视区。
- 缓冲区：在可视区上下额外多渲染几个节点（Buffer），防止快速滚动时白屏。

常见坑：

- 列表项高度不固定时的处理（需要预估高度并在渲染后更新）。
- 滚动过快导致的白屏闪烁优化。

追问：

- 如何处理高度动态变化的列表项？
 - 这种方案在移动端会有什么兼容性问题吗？
-

三面（架构与综合能力）

1. 如果让你从零搭建一个前端监控系统，你会如何设计它的架构？

难度：★★★★

考点：系统设计、全链路思维

答案要点：

前端监控系统应包含数据采集、数据上报、数据处理和可视化监控四个核心模块。

- 数据采集：通过 SDK 自动捕获 JS 错误、Promise 异常、资源加载失败、性能指标（Vitals）及用户行为轨迹。
- 上报策略：采用 navigator.sendBeacon 保证页面卸载时也能成功上报，并支持批量上报与节流。
- 数据处理：后端对原始日志进行清洗、脱敏，并利用 SourceMap 还原错误堆栈。
- 告警体系：设置阈值告警，通过钉钉/邮件通知相关负责人，并提供多维度的看板展示。

常见坑：

- 监控 SDK 自身报错导致业务页面崩溃。
- 上报流量过大对业务服务器造成压力。

追问：

- 如何实现无痕埋点？
- 如何保证监控数据的准确性和不丢失？

2. 在过去的项目中，你遇到的最具有挑战性的技术问题是什么？你是如何解决的？

难度：★★★★

考点：问题解决能力、复盘深度

答案要点：

这是一个开放性问题，重点在于体现你发现问题、分析原因、寻找方案、落地执行及最后复盘的闭环能力。

- 问题背景：清晰描述当时的技术背景、业务压力和具体的技术痛点。
- 分析过程：使用了哪些工具（如 Chrome DevTools, Wireshark, 性能分析器）定位到了核心原因。
- 方案对比：当时考虑了哪几种方案，为什么最终选择了这一种（权衡利弊）。
- 最终结果：量化的改进指标（如首屏时间降低 50%，内存泄漏消失等）。

常见坑：

- 描述过于琐碎，没有体现出技术深度。
- 只讲结果，不讲分析问题的逻辑过程。

3. 聊聊你对前端工程化的理解，你认为目前团队最需要提升的工程化环节是什么？

难度：★★★

考点：工程化视野

答案要点：

前端工程化不仅仅是打包工具，而是涵盖了从开发、测试、部署到运维的全生命周期规范化。

- 规范化：代码规范（Lint）、提交规范（Commitlint）、目录结构规范。
- 自动化：CI/CD 流动线、自动化测试（单元测试、E2E）、自动发布。
- 模块化/组件化：提升代码复用率，建立团队私有组件库。
- 提效工具：脚手架工具、Mock Server、低代码平台等。

常见坑：

- 把工程化等同于 Webpack 配置。
- 引入了过于复杂的流程却没能真正提升开发效率。

追问：

- 如何衡量一个工程化方案的成功与否？

4. 面对一个大型遗留项目（Legacy Project），如果需要进行技术栈升级，你会制定怎样的策略？

难度：★★★

考点：架构演进、风险控制

答案要点：

技术栈升级必须遵循“渐进式”原则，严禁推倒重来，确保业务连续性。

- 现状评估：梳理核心业务链路，评估旧代码的耦合度和测试覆盖率。
- 灰度方案：采用微前端架构或多页应用共存模式，让新旧代码在同一系统中运行。

- 适配层设计：编写适配器处理新旧框架间的通信和状态共享。
- 节奏控制：先从非核心模块开始试点，积累经验后再逐步覆盖核心模块，并做好回滚预案。

常见坑：

- 盲目追求新技术导致项目进度失控。
- 忽略了旧数据的迁移和兼容性问题。

追问：

- 如何说服业务方支持这种纯技术性的重构？

5. 你是如何保持技术竞争力的？最近在关注哪些前端领域的新趋势？

难度：★★

考点：学习动力、技术敏感度

答案要点：

面试官希望看到候选人有自驱动的学习习惯和对行业趋势的敏锐洞察。

- 学习路径：阅读源码、参与开源项目、撰写技术博客、关注 RFC 提案。
- 关注领域：如 WebAssembly 在高性能计算的应用、Serverless 对前端边界的扩展、Rust 构建工具（如 Turbopack/Rspack）对生态的重塑。
- 实践思考：不仅是看，更重要的是思考这些新技术如何解决当前业务中的实际问题。

常见坑：

- 罗列一堆名词但说不出其背后的原理或解决的痛点。
- 表现出对基础知识的轻视，只追逐热点。

6. 谈谈你对前端团队协作和 Code Review 的看法。

难度：★★

考点：团队协作、软技能

答案要点：

良好的协作机制是高质量交付的保证，Code Review 是知识传递和质量把控的重要手段。

- CR 目的：发现潜在 Bug、统一代码风格、促进团队成员间的技术交流。
- 协作流程：明确的 PR 流程、完善的文档沉淀（RFC/技术方案设计）。
- 沟通技巧：CR 时对事不对人，给出建设性意见而非单纯的批评。
- 自动化辅助：利用工具自动检查基础规范，让人类专注于逻辑和架构审查。

常见坑：

- 把 CR 变成一种形式主义，或者变成权力斗争的工具。
- 缺乏明确的文档导致沟通成本极高。